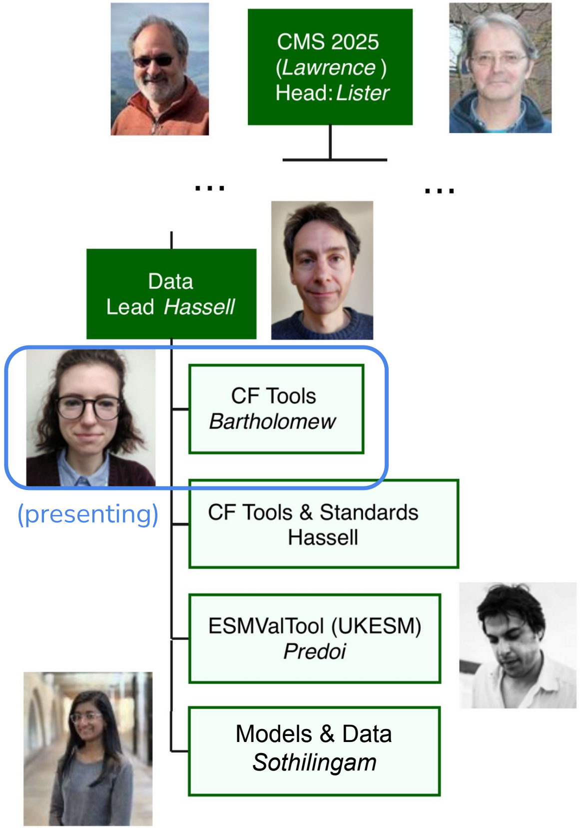
The NCAS-CMS Data Team: Big Questions



1. How can we ensure earth system data is **self-describing and interoperable**, so researchers can easily *find, understand and combine* diverse datasets in use (spanning modelling, obs, reanalysis etc.)?
2. How can we best employ **modern computing including HPC and cloud technologies** to support *efficiently* working with huge datasets?
3. How can we build *and maintain* a powerful, flexible and robust **data analysis and visualisation ecosystem** by embedding open standards and best practices into climate science workflows? And (how) should we use generative AI to help us keep up with the pace?

In short, how can we, for earth and related science:

1. *Make data access and interpretation simplest?*
2. *Keep up with the scale of modern research & infrastructure?*
3. *Provide the best data tools to facilitate research?*

University of Reading

Some recent results and near future work*

*Focusing on what I have contributed to, but the CMS Data Team have much more to showcase!

Not at all domain-specific: *no knowledge of CF-netCDF*

Note: only including Python tools/ecosystem

Fully domain-specific: *complete knowledge of CF-netCDF*

`numpy` `(or dask)` — `netCDF4-python` — `xarray` — `cf_xarray` — `iris` — `cf-python` (and friends †)

CF MetaData

dask

- **At scale (and e.g. for generative AI to follow), metadata is crucial**. We centre around the official CF Data Model and keep up with the **CF Conventions metadata standard** (yearly release). Example recent new CF aspects we support include *UGRID meshes, HEALPix grids, aggregation, and improved treatment of time.*
- **Data reading and writing flexibility: remote data access is paramount in 2020+**. Recent work has focused on supporting (STAC etc.) catalog and data stores use; reading and writing of cloud-native formats (e.g. Zarr, Kerchunk) with flexible backend support; and conversion to and from `xarray`.
- **Performance is always considered**, including how we compare there with other tools (see above). **We use Dask under-the-hood** for our data manipulations, allowing for lazy evaluation, flexible chunking, out-of-core computation, and parallel execution.
- **Built-in compliance checking**: early work is complete to enable some CF compliance checking natively within our `cf.read` functionality, which we hope to build on soon.

† Friends include: `cfdm`, `cf-plot`, and the shiny new tools being prepared for the near future, `xnetcdf` (consolidating and improving read/write) and `xconv2` (GUI, building on all of these libraries)

University of Reading